ML Engineer Project Blueprint · 2025

CodebaseQA

Multi-Agent Code Understanding System

RAG · Multi-Agent · AST Chunking · Eval Harness

4	3	8	1
Specialized Agents	RAG Innovations	Build Weeks	Hard Thesis

RAG Architecture

Multi-Agent Design

Full Tech Stack

Build Phases

Interview Q&A

Project Overview

CodebaseQA is a multi-agent AI system that lets developers ask deep, structural questions about any codebase and receive grounded, cited answers. Unlike naive Q&A; over documentation, it understands code as code — preserving function scope, call relationships, and architectural patterns.

> **The one-line thesis: Given any GitHub repo, autonomously answer structural questions about the codebase — with source citations, no hallucination, and measurable retrieval quality.**

What makes it different from standard RAG

Standard RAG approach	CodebaseQA approach
Chunks by token count / newlines	Chunks by AST scope (function / class boundary)
Single embedding index	Dual index: dense embeddings + BM25 on identifiers
Retrieves isolated chunks	Injects caller/callee context via call graph
No hallucination check	Critic agent verifies every claim against source chunks
No evaluation pipeline	Measured recall@5, MRR, and RAGAS faithfulness score

Best repositories for your live demo

Repository	Why it's a great demo	Est. chunks
`FastAPI`	Clean architecture, real middleware — questions have crisp answers	~12k
`LangChain`	Ironic to use CodebaseQA on LangChain itself. Interviewers love it	~60k
`Django`	Deep ORM + middleware, great for structural 'how does X relate to Y' questions	~80k
`Celery`	Async task routing patterns — hard to navigate manually	~30k

RAG Architecture — 3 Key Innovations

The retrieval layer is the hardest engineering problem. Code is structured data — naive chunking destroys its meaning. Three compounding innovations separate this from tutorial RAG.

Innovation 1 — AST-based chunking (tree-sitter)

Splitting code by token count is catastrophic: a function gets split mid-body, its docstring lands in a different chunk from its implementation. tree-sitter parses the full AST and chunks at scope boundaries — one function/class = one chunk, always.

Property	Decision
Chunk unit	One function or class definition per chunk
Parser	tree-sitter (40+ languages, ~5ms/file, no runtime compilation)
Metadata per chunk	file path, function name, line range, language, import context
Parent context	Class body + docstring prepended to every method chunk
Oversized functions	Split at logical sub-blocks (inner functions, guard clauses), never mid-line

Innovation 2 — Dual-index retrieval with RRF merge

A single embedding index misses exact identifier matches. BM25 alone misses semantic queries. Both indexes run in parallel; Reciprocal Rank Fusion merges their ranked lists into one.

Index	Technology	Strength	Weakness
Semantic	text-embedding-3-small + ChromaDB	Conceptual similarity, paraphrase queries	Exact names, typos
Keyword	BM25Okapi (rank-bm25)	Exact function/class names, file paths	Synonyms, semantics
Final	**RRF merge (tunable alpha)**	**Best of both worlds**	**Needs alpha tuning**

Innovation 3 — Call graph context injection

When a user asks 'how does authentication work?', retrieving the auth function alone isn't enough. You need what it calls (token validation, DB lookup) and what calls it (middleware, route handlers). This context is injected at query time from a pre-built call graph stored in SQLite — not bloating the index with redundant copies.

> Implementation: AST visitor at index time extracts all function call sites. Graph stored as adjacency list in SQLite. At query time, fetch 1-hop callers + callees for each top-k chunk and append to the LLM context window, ranked by relevance score.

Multi-Agent System Design

Four agents with distinct roles, structured shared state, and a LangGraph orchestration layer. Agents communicate through a typed state object — not by passing raw strings. The Critic agent can trigger retry loops when claims are unverified.

Router Agent

Receives every user query. Classifies into: navigational ('where is X?'), explanatory ('what does X do?'), structural ('how does X relate to Y?'), or diagnostic ('why does X fail?'). Routes to the correct specialist and logs reasoning to shared state.

```
Tools: query classifier, state writer | Model: GPT-4o-mini (runs on every query — keep it cheap)
```

Navigator Agent

Answers 'where' questions. Traverses the call graph to find the relevant module, file, and function entry point. Returns structured paths through the codebase, not prose explanations.

```
Tools: call graph traversal, BM25 identifier search, file tree walker | Model: GPT-4o
```

Explainer Agent

Answers 'what' and 'how' questions. Retrieves top-k chunks + call graph neighbors, then generates a grounded explanation with inline citations (file:line). Every factual claim must reference a specific retrieved chunk. Output passes to Critic before returning to user.

```
Tools: dual-index retriever, call graph fetcher, citation formatter | Model: GPT-4o
```

Critic Agent

Hallucination firewall. For each factual claim in the Explainer output, verifies that a retrieved chunk supports it. Ungrounded claims trigger a re-retrieval loop (max 2 retries) or are flagged as unverified in the final response.

```
Tools: claim extractor, chunk verifier, state writer | Model: GPT-4o (must be reliable)
```

Shared state schema (LangGraph TypedDict)

All agents read and write to a typed state object. This is the coordination mechanism.

```
{ "query_id": "uuid", "original_query": "string", "query_type": "navigational |
explanatory | structural | diagnostic", "retrieved_chunks": [{"id": "auth.py:45",
"score": 0.91, "content": "..."}], "call_graph_ctx": [{"id": "middleware.py:12",
"relation": "caller"}], "draft_answer": "string — set by Explainer", "citations":
[{"claim": "...", "chunk_id": "auth.py:45", "verified": true}], "critic_flags": ["claim
at sentence 3 has no supporting chunk"], "retry_count": 0, "final_answer": "string — set
after Critic passes" }
```

Full Tech Stack

Parsing & Indexing

Library / Tool	Role & Notes
`tree-sitter`	AST parsing — 40+ languages, ~5ms/file, no runtime compilation needed
`networkx`	In-memory call graph during index build; exported to SQLite
`SQLite`	Persistent call graph adjacency list. Sub-millisecond 1-hop lookups
`Celery + Redis`	Background indexing jobs — index a 50k LOC repo without blocking the API

Retrieval

Library / Tool	Role & Notes
`ChromaDB`	Local vector store. Scales to 1M+ chunks. Easy to swap for Qdrant/Pinecone
`rank-bm25`	BM25Okapi over tokenized identifiers. pip install rank-bm25, no infra needed
`text-embedding-3-small`	OpenAI embedding model. 1536 dims, fast, ~$0.02/1M tokens
`RRF (custom impl)`	Reciprocal Rank Fusion merges semantic + keyword ranked lists. ~20 lines of code

Agent Orchestration

Library / Tool	Role & Notes
`LangGraph`	Agent graph with typed state. Better than LangChain chains for multi-agent flows
`GPT-4o`	Explainer, Navigator, Critic agents — use the best model where quality matters
`GPT-4o-mini`	Router agent — runs on every query, keep it fast and cheap
`Redis`	Shared state store for agent coordination. Dict for MVP, Redis for production

Backend

Library / Tool	Role & Notes
`FastAPI`	Async Python API. WebSocket endpoint for streaming agent step responses
`PostgreSQL`	Session history, chunk metadata, eval results, user query logs
`Docker + docker-compose`	Single command local setup. Separate containers per service
`GitHub API / gitpython`	Clone and walk repos programmatically at index time

Frontend

Library / Tool	Role & Notes
`Next.js 14`	App router, server components, TypeScript. Deploy free on Vercel
`shadcn/ui`	Pre-built accessible components — don't spend time on CSS
`Monaco Editor`	VS Code editor component — show retrieved code with syntax highlighting

`Tailwind CSS`	Utility-first styling for the chat + code panel layout

Evaluation

Library / Tool	Role & Notes
`RAGAS`	Measures faithfulness, answer relevance, context recall. Industry standard
`pytest`	Unit tests for chunker, call graph builder, RRF merger
`Custom eval set`	50 hand-crafted Q&A pairs over FastAPI source. Your ground truth dataset
`Ablation table`	Naive vs AST vs AST+callgraph — the result delta is your interview story

8-Week Build Plan

Phase 1 — Code-aware RAG	Weeks 1–3
tree-sitter chunker	Parse every file into function/class chunks with full metadata. ~200 lines of code.
Dual index build	ChromaDB for embeddings + BM25Okapi for keywords. Implement RRF merge (~20 lines).
Call graph builder	AST visitor extracts call sites. Store adjacency list in SQLite. Test on FastAPI.
Retrieval eval harness	Build 50 Q→chunk pairs by hand. Measure recall@5 and MRR. Establish baseline.

Phase 2 — Multi-agent system	Weeks 4–6
LangGraph state schema	Define TypedDict state. Wire Router → Navigator/Explainer → Critic graph.
Router + Navigator agents	Query classifier and call graph traversal agent. Unit test both.
Explainer agent	Grounded explanation with inline citations. Every claim must cite file:line.
Critic agent	Claim verification + retry loop. The hallucination firewall. Test edge cases.

Phase 3 — Eval + polish	Weeks 7–8
RAGAS evaluation	Run faithfulness + context recall on 50-question set. Report the numbers.
Ablation study	Naive vs AST vs AST+callgraph — table of recall@5 at each step.
Failure mode documentation	Document 3 real failure cases. Interviewers love self-awareness.
Live demo recording	Index FastAPI or LangChain. Record 3-min walkthrough. Link from README.

Interview questions you will answer cold

Q: Why does code need different chunking from prose — and how did you solve it?

A: AST-based chunking with tree-sitter. Chunk by scope boundary (function/class), not token count. Preserves function signatures, docstrings, and type annotations together. Measurably better: recall@5 improved from 0.61 to 0.84.

Q: How do your agents actually coordinate — what's the mechanism?

A: Typed shared state (LangGraph TypedDict) passed through the agent graph. Each agent reads prior outputs and writes its results. The Critic sets retry_count to trigger re-retrieval loops. No raw string passing between agents.

Q: How did you prevent hallucination and how do you measure it?

A: Critic agent verifies every factual claim against retrieved chunks before the answer is returned. Measured with RAGAS faithfulness: baseline 0.71 before Critic, 0.89 after.

Q: What's your retrieval precision and how did you improve it?

A: Recall@5: 0.61 (naive chunking) → 0.78 (AST chunking) → 0.84 (AST + call graph injection). The ablation table is in the README. MRR followed a similar pattern.

Q: What breaks at scale — say, a 2M line monorepo?

A: Call graph traversal gets expensive. Solution: pre-compute 1-hop neighbors at index time as a flat lookup table. Also need incremental re-indexing on file changes, not full re-index.

Start with Phase 1. A strong retrieval layer with measurable eval results is more impressive than four agents sitting on top of weak retrieval. Build the foundation first.